



CVRP: Método Exacto vs. Particle Swarm Optimization

Trabajo de investigación — Informes 1 y 2

Cristopher Angulo Ahumada

Optimización en Ingeniería ■ Profs. Dra. Mónica Villanueva I. y Dr. Mario Inostroza P.

Julio de 2026

Método exacto: Laporte & Nobert (1983) + cortes RCI ■ Metaheurística: PSO con representación SR-1 (Ai & Kachitvichyanukul, 2009)

- 1 Introducción
- 2 Descripción del problema
- 3 Modelamiento matemático
- 4 Estado del arte
- 5 Solución con método clásico
- 6 Solución con metaheurística
- 7 Comparación de resultados
- 8 Conclusiones
- 9 Referencias

Introducción

Introducción: por qué importa el ruteo de vehículos

Motivación

El transporte es uno de los mayores costos de una operación logística. Una planificación deficiente de rutas incrementa **kilómetros recorridos**, combustible, horas-hombre y uso de flota. El **CVRP** es el modelo base para decidir **qué reparte cada vehículo** y **en qué orden**, respetando su capacidad.

Dos contextos de aplicación (Golden et al., 2002)

- **Distribución de bebidas y alimentos:** un centro de distribución abastece puntos de venta con flota propia; consolidar carga reduce despachos y costo por entrega.
- **Recolección de residuos sólidos:** camiones recorren sectores urbanos y deben volver al depósito antes de superar su capacidad de compactación (Beliën et al., 2014).



Del reparto urbano de última milla
a la logística de distribución.

Introducción: objetivo del trabajo

Idea clave

Resolver el **CVRP** con dos enfoques de naturaleza distinta y **contrastarlos** sobre un conjunto común de instancias: un **método clásico exacto** (Informe 1) y una **metaheurística** (Informe 2).

Informe 1 — método clásico

Branch and Bound con cortes de capacidad (RCI), formulación de Laporte & Nobert (1983), resuelto con PuLP + COIN-OR CBC. **Certifica optimalidad.**

Informe 2 — metaheurística

Particle Swarm Optimization con representación SR-1 (Ai & Kachitvichyanukul, 2009), parametrizada con Optuna. **Escala**, sin certificado.

Se comparan en **calidad de solución** (GAP respecto al óptimo o BKS), **tiempo de ejecución** y **escalabilidad**.

Descripción del problema

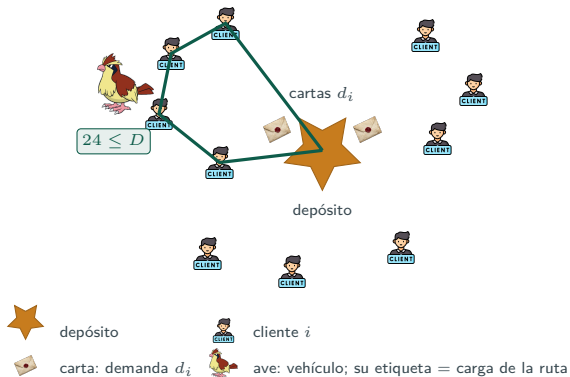
El CVRP en una imagen



Los datos

Cada **cliente** espera su carta, que pesa d_i ; hay un **depósito** y m **aves** con la misma **carga máxima** D (aquí $m = 3$, $D = 30$).

El CVRP en una imagen



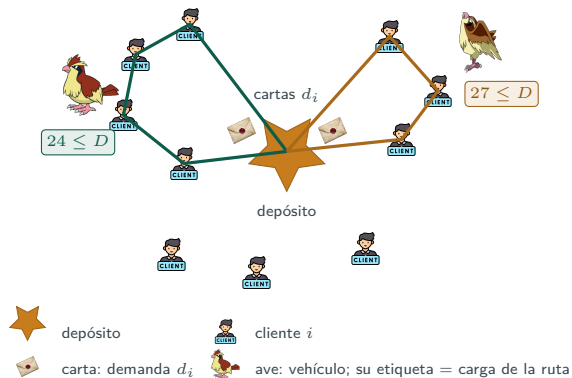
Los datos

Cada **cliente** espera su carta, que pesa d_i ; hay un **depósito** y m **aves** con la misma **carga máxima** D (aquí $m = 3$, $D = 30$).

La decisión

Repartir los clientes entre las aves y fijar el **orden** de visita de cada una.

El CVRP en una imagen



Los datos

Cada **cliente** espera su carta, que pesa d_i ; hay un **depósito** y m **aves** con la misma **carga máxima** D (aquí $m = 3$, $D = 30$).

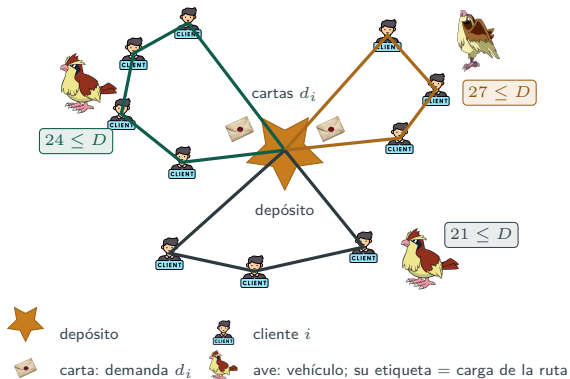
La decisión

Repartir los clientes entre las aves y fijar el **orden** de visita de cada una.

Las restricciones

Ninguna ave carga más de D ; cada cliente lo atiende **una sola** ave. Objetivo: **mínima distancia total volada**.

El CVRP en una imagen



Los datos

Cada **cliente** espera su carta, que pesa d_i ; hay un **depósito** y m **aves** con la misma **carga máxima** D (aquí $m = 3$, $D = 30$).

La decisión

Repartir los clientes entre las aves y fijar el **orden** de visita de cada una.

Las restricciones

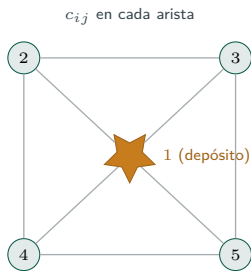
Ninguna ave carga más de D ; cada cliente lo atiende **una sola** ave. Objetivo: **mínima distancia total volada**.

Idea clave

Hay que decidir **a la vez** qué **clientes atiende cada ave** (partición, limitada por el peso de las cartas) y **en qué orden los visita** (secuenciación, que fija la distancia). Ambas decisiones están **acopladas**: por eso el CVRP es NP-hard.

Modelamiento matemático

Modelamiento sobre grafos



Grafo completo no dirigido $G = (V, E)$.

En el **anexo** del final: glosario de símbolos y el modelo entero completo (variables x_{ij} , restricciones de grado y cortes RCI).

Datos

$V = \{1, \dots, n\}$ es el conjunto de **nodos**: el 1 es el depósito y $2, \dots, n$ los clientes. E es el de **aristas**, una por cada par de nodos. Cada arista $\{i, j\}$ tiene costo c_{ij} ; cada cliente i , demanda $d_i \geq 0$. La flota son m vehículos, todos de capacidad D .

Función objetivo y restricciones

Una solución es un conjunto de rutas $\mathcal{R}$; cada $r \in \mathcal{R}$ es el ciclo de aristas que recorre un vehículo desde el depósito y de vuelta. Se minimiza la distancia total de la flota:

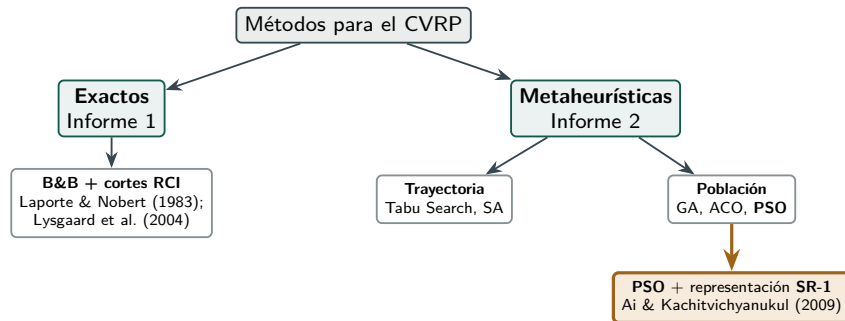
$$\text{mín } z = \sum_{r \in \mathcal{R}} \sum_{\{i,j\} \in r} c_{ij}$$

sujeto a que las rutas **particionen** a los clientes y respeten la capacidad:

$$\sum_{i \in r} d_i \leq D \quad \forall r \in \mathcal{R}.$$

Estado del arte

Estado del arte: dónde se sitúa este trabajo



Las metaheurísticas renuncian a la **garantía de optimalidad** a cambio de **escalar** a instancias industriales (Tan & Yeh, 2021). Revisión específica de PSO para ruteo: Marinakis et al. (2018).

Estado del arte: linaje de PSO y las tres vías de adaptación



1. Valor real

La partícula sigue continua; un **decodificador** la traduce a rutas.

► Vía adoptada (SR-1)

2. PSO discreto

Redefine la velocidad como secuencias de *swaps* (Chen et al., 2006).

Descartada

3. Híbridos

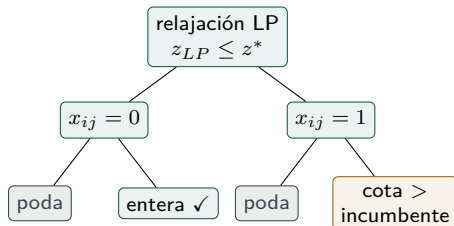
PSO + ACO, o *clustering* + PSO (Kao et al., 2012; Son & Tan, 2021).

Descartada

Se eligió la vía 1 porque **preserva las ecuaciones canónicas de PSO** y permite aislar su comportamiento propio, sin la infraestructura extra de un híbrido.

Solución con método clásico

La idea: Branch and Bound con cortes RCI



Laporte & Nobert (1983)

- Relajación lineal → cota inferior.
- **Ramificar** sobre variables fraccionarias; **podar** ramas dominadas.

Cortes RCI

Los *rounded capacity inequalities* se agregan **iterativamente** al detectar subrutas que incumplen la capacidad: fortalecen la cota sin agrandar el modelo de entrada.

Idea clave

Si el árbol se agota: **óptimo certificado**. Ese certificado es lo que la metaheurística no puede dar.

Pseudocódigo: generación iterativa de cortes RCI

```
1: procedure EXACTO-RCI(instancia)
2:   Construir grafo y matriz  $c_{ij}$ 
3:   Formular modelo entero
4:   repeat
5:     Resolver el modelo con CBC
6:     Extraer las aristas activas
7:     Buscar  $S$  con RCI incumplida
8:     if existe alguna RCI incumplida then
9:       Agregar ese corte RCI
10:    end if
11:  until no queden RCI incumplidas
12:  Devolver rutas (óptimo certificado)
13: end procedure
```

El corte RCI

Queda incumplido si un subconjunto S forma un subtour sin depósito, o si su carga excede D :

$$\sum_{i < j \in S} x_{ij} \leq |S| - \left\lceil \frac{\sum_{i \in S} d_i}{D} \right\rceil$$

Por qué iterar

Las restricciones de capacidad son **exponenciales**: se agregan **solo cuando la solución las incumple**. Pero cada corte obliga a **reoptimizar**: en A-n33-k5 fueron **384 iteraciones y 2 846 cortes**.

Resultados del método exacto (Informe 1)

Instancia	n	z	Iter.	Cortes	Tiempo (s)
E-n13-k4	13	247	15	47	0,9
E-n22-k4	22	375	13	84	0,8
E-n23-k3	23	569	16	135	1,2
A-n32-k5	32	784	99	788	576,1
A-n33-k5	33	667 [†]	384	2 846	17 096,4

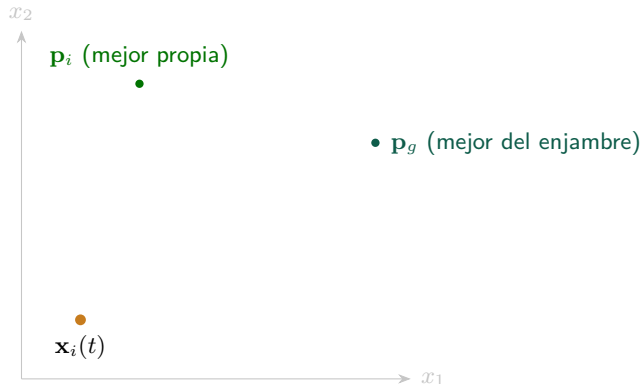
[†] Sin certificado de optimalidad (límite de 60s por resolución del modelo entero); el óptimo real es 661.

Idea clave

La tratabilidad se pierde de forma **abrupta**, no gradual: **un solo cliente más** ($n = 32 \rightarrow 33$) lleva de 10 minutos a **4,7 horas sin certificado**. Más allá de ese tamaño, el exacto queda fuera de alcance práctico.

Solución con metaheurística

PSO en 30 segundos: tres “tirones” que se suman



Actualización canónica

$$\mathbf{v}_i \leftarrow w\mathbf{v}_i + c_1\mathbf{r}_1 \otimes (\mathbf{p}_i - \mathbf{x}_i) + c_2\mathbf{r}_2 \otimes (\mathbf{p}_g - \mathbf{x}_i)$$

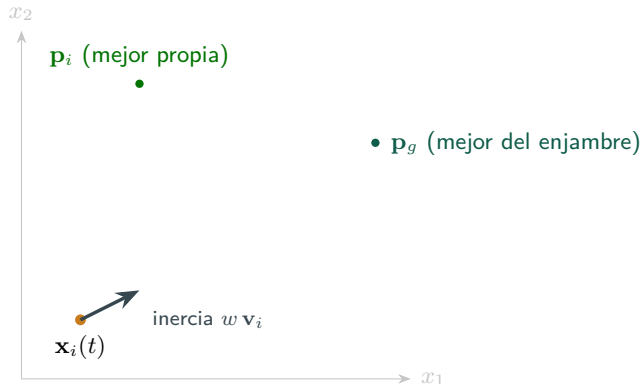
$$\mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{v}_i$$

- Inercia: sigue el rumbo.
- Cognitivo: hacia su recuerdo.
- Social: hacia el mejor del enjambre.

Idea clave

El progreso emerge de **compartir información**, no de un individuo.

PSO en 30 segundos: tres “tirones” que se suman



Actualización canónica

$$\mathbf{v}_i \leftarrow w\mathbf{v}_i + c_1\mathbf{r}_1 \otimes (\mathbf{p}_i - \mathbf{x}_i) + c_2\mathbf{r}_2 \otimes (\mathbf{p}_g - \mathbf{x}_i)$$

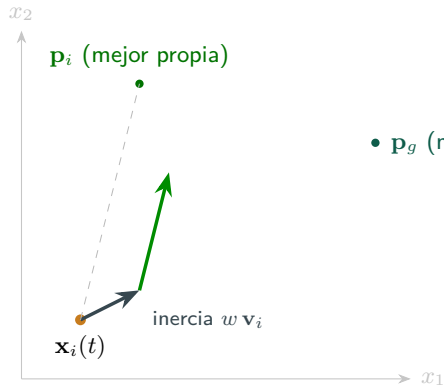
$$\mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{v}_i$$

- Inercia: sigue el rumbo.
- Cognitivo: hacia su recuerdo.
- Social: hacia el mejor del enjambre.

Idea clave

El progreso emerge de **compartir información**, no de un individuo.

PSO en 30 segundos: tres “tirones” que se suman



- p_g (mejor del enjambre)

Actualización canónica

$$\mathbf{v}_i \leftarrow w\mathbf{v}_i + c_1\mathbf{r}_1 \otimes (\mathbf{p}_i - \mathbf{x}_i) + c_2\mathbf{r}_2 \otimes (\mathbf{p}_g - \mathbf{x}_i)$$

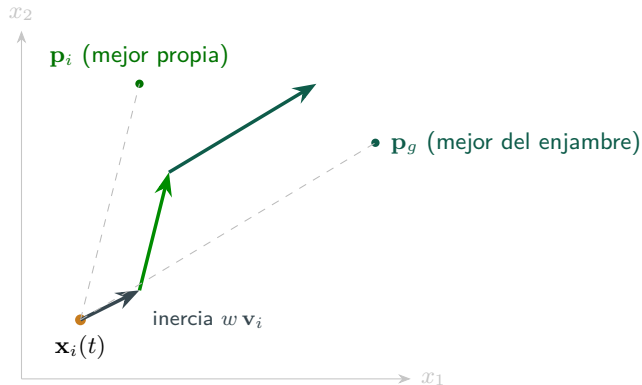
$$\mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{v}_i$$

- Inercia: sigue el rumbo.
- Cognitivo: hacia su recuerdo.
- Social: hacia el mejor del enjambre.

Idea clave

El progreso emerge de **compartir información**, no de un individuo.

PSO en 30 segundos: tres “tirones” que se suman



Actualización canónica

$$\mathbf{v}_i \leftarrow w\mathbf{v}_i + c_1\mathbf{r}_1 \otimes (\mathbf{p}_i - \mathbf{x}_i) + c_2\mathbf{r}_2 \otimes (\mathbf{p}_g - \mathbf{x}_i)$$

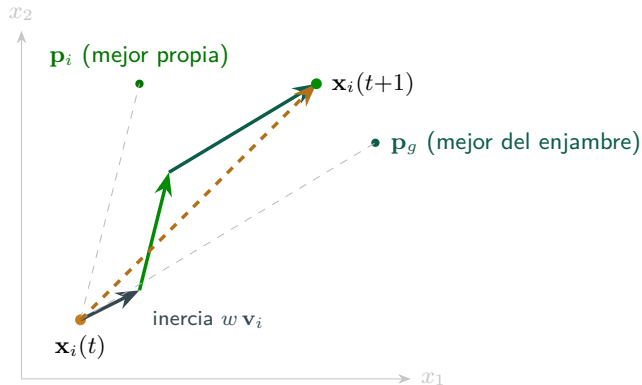
$$\mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{v}_i$$

- ■ Inercia: sigue el rumbo.
- ■ Cognitivo: hacia su recuerdo.
- ■ Social: hacia el mejor del enjambre.

Idea clave

El progreso emerge de compartir información, no de un individuo.

PSO en 30 segundos: tres “tirones” que se suman



Actualización canónica

$$\mathbf{v}_i \leftarrow w\mathbf{v}_i + c_1\mathbf{r}_1 \otimes (\mathbf{p}_i - \mathbf{x}_i) + c_2\mathbf{r}_2 \otimes (\mathbf{p}_g - \mathbf{x}_i)$$

$$\mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{v}_i$$

- Inercia: sigue el rumbo.
- Cognitivo: hacia su recuerdo.
- Social: hacia el mejor del enjambre.

Idea clave

El progreso **emerge de compartir información**, no de un individuo.

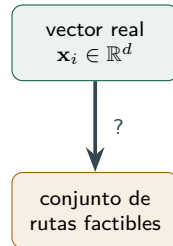
El desafío: PSO es continuo, el CVRP es combinatorio

Lo que PSO sabe hacer

Mover **vectores reales**: $\mathbf{x}_i \in \mathbb{R}^d$, sumar velocidades, interpolar posiciones.

Lo que el CVRP necesita

Un conjunto de **rutas**: no existe una noción natural de “sumar una velocidad” a una partición de clientes.



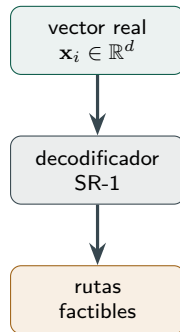
Idea clave

El desajuste entre un algoritmo **continuo** y un problema **combinatorio** es la decisión de diseño central: hay que definir cómo un vector real representa una solución del CVRP y cómo recuperar las rutas a partir de él.

La solución: representación SR-1 (Ai & Kachitvichyanukul, 2009)

La idea

- La partícula **sigue siendo un vector real** de $(n - 1) + 2m$ dimensiones.
- Un **decodificador** la traduce a rutas factibles en capacidad.
- Las ecuaciones de PSO **no se modifican**.



Ojo con la analogía

Una partícula **no** es un ave. Cada partícula codifica **una solución completa**: qué cliente atiende *cada* ave y en qué orden. En A-n32-k5 ($m=5$), **una** partícula de 41 dimensiones decodifica a las **5 rutas** a la vez; el enjambre son $N=100$ **planes de reparto alternativos** compitiendo entre sí.

Decodificación SR-1 paso a paso (ejemplo del paper: 6 clientes, 2 vehículos)

	claves de prioridad (clientes 1–6)						puntos de ref. (veh. 1 y 2)			
	1	2	3	4	5	6				
Partícula:	0.4	0.2	0.9	0.5	0.6	0.1	0.8	1.5	0.3	1.2

Decodificación SR-1 paso a paso (ejemplo del paper: 6 clientes, 2 vehículos)

	claves de prioridad (clientes 1–6)						puntos de ref. (veh. 1 y 2)			
	1	2	3	4	5	6				
Partícula:	0.4	0.2	0.9	0.5	0.6	0.1	0.8	1.5	0.3	1.2

1. Ordenar claves ascendente (SPV): prioridad = 6, 2, 1, 4, 5, 3

Decodificación SR-1 paso a paso (ejemplo del paper: 6 clientes, 2 vehículos)

	claves de prioridad (clientes 1–6)						puntos de ref. (veh. 1 y 2)			
	1	2	3	4	5	6				
Partícula:	0.4	0.2	0.9	0.5	0.6	0.1	0.8	1.5	0.3	1.2

1. Ordenar claves ascendente (SPV):

prioridad = 6, 2, 1, 4, 5, 3

2. Referencias de vehículos:

$V_1 = (0,8; 0,3)$ $V_2 = (1,5; 1,2) \Rightarrow$ cada cliente prefiere al más cercano

Decodificación SR-1 paso a paso (ejemplo del paper: 6 clientes, 2 vehículos)

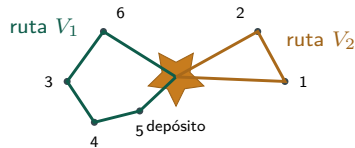
	claves de prioridad (clientes 1–6)						puntos de ref. (veh. 1 y 2)			
	1	2	3	4	5	6				
Partícula:	0.4	0.2	0.9	0.5	0.6	0.1	0.8	1.5	0.3	1.2

1. Ordenar claves ascendente (SPV): $\text{prioridad} = 6, 2, 1, 4, 5, 3$
2. Referencias de vehículos: $V_1 = (0,8; 0,3)$ $V_2 = (1,5; 1,2) \Rightarrow$ cada cliente prefiere al más cercano
3. Insertar en ese orden, al menor costo, respetando capacidad; 2-opt tras cada inserción.

Decodificación SR-1 paso a paso (ejemplo del paper: 6 clientes, 2 vehículos)

	claves de prioridad (clientes 1–6)						puntos de ref. (veh. 1 y 2)			
	1	2	3	4	5	6				
Partícula:	0.4	0.2	0.9	0.5	0.6	0.1	0.8	1.5	0.3	1.2

1. Ordenar claves ascendente (SPV): prioridad = 6, 2, 1, 4, 5, 3
2. Referencias de vehículos: $V_1 = (0,8; 0,3)$ $V_2 = (1,5; 1,2) \Rightarrow$ cada cliente prefiere al más cercano
3. Insertar en ese orden, al menor costo, respetando capacidad; 2-opt tras cada inserción.



V_1 : 0-6-3-4-5-0

V_2 : 0-2-1-0

► Rutas factibles en capacidad por construcción: en las **279** corridas, ningún cliente quedó sin asignar.

Pseudocódigo: PSO para el CVRP (representación SR-1)

```
1: procedure PSO-CVRP(instancia, parámetros)
2:   Inicializar  $\mathbf{x}_i \sim U(0, 1)^d$ ,  $\mathbf{v}_i \leftarrow \mathbf{0}$ 
3:   for cada partícula  $i$  do
4:     Decodificar  $\mathbf{x}_i$  a las  $m$  rutas; evaluar
5:      $\mathbf{p}_i \leftarrow \mathbf{x}_i$ 
6:   end for
7:    $\mathbf{p}_g \leftarrow \arg \min_i \text{aptitud}(\mathbf{p}_i)$ 
8:   for  $t \leftarrow 1 \dots T$  do
9:      $w \leftarrow w_0 + \frac{t}{T}(w_f - w_0)$ 
10:    for cada partícula  $i$  do
11:      Actualizar  $\mathbf{v}_i$ ,  $\mathbf{x}_i$ ; acotar a  $v_{\max}$  y  $[0, 1]$ 
12:      Decodificar  $\mathbf{x}_i$ ; evaluar aptitud
13:      si mejora: actualizar  $\mathbf{p}_i$  y  $\mathbf{p}_g$ 
14:    end for
15:  end for
16:  Devolver rutas de  $\mathbf{p}_g$ 
17: end procedure
```

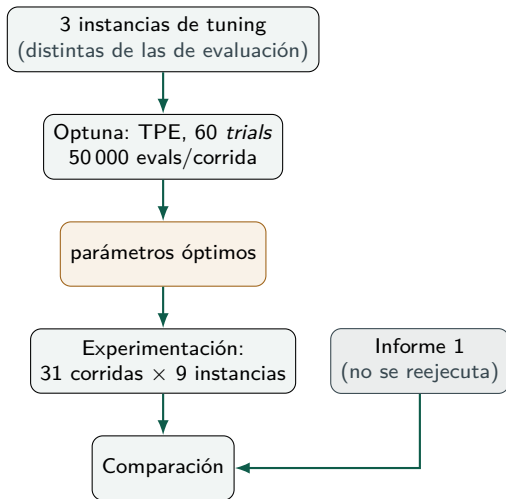
Dónde entra SR-1

Las líneas de **decodificación** (4 y 12) son las **únicas** específicas del CVRP: ahí el vector continuo se convierte en las m rutas. **Todo lo demás es PSO canónico, sin modificar.**

Contabilidad del presupuesto

Cada decodificación es **una evaluación** de la función objetivo. Con $N=100$ y $T=500$:
 $100 \times 500 = 50\,000$ evaluaciones, el presupuesto fijado para todas las instancias.

Parametrización con Optuna y diseño experimental



Parámetros encontrados

N	partículas	100
T	iteraciones	500
w	inercia	0,91 → 0,28
c_1, c_2	cognitivo/social	2,40 / 1,18
$v_{\text{máx}}$	tope velocidad	0,38

Instancias (CVRPLIB, Sets A y E)

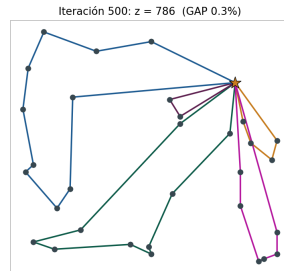
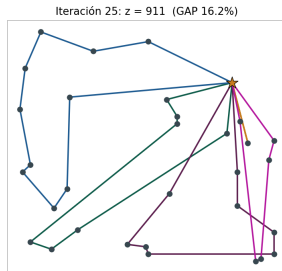
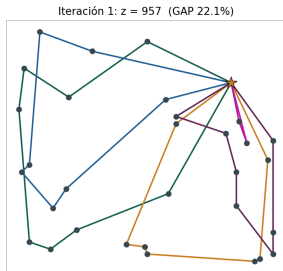
- **5 compartidas** con el Informe 1 ($n = 13$ a 33).
- **4 mayores** ($n = 60$ a 101), fuera del alcance del exacto.

Idea clave

Separar **tuning** de **evaluación** evita el sobreajuste. GAP de tuning: 8,54 %.

Comparación de resultados

El enjambre aprendiendo rutas (A-n32-k5, corrida real)



Iteración 1: rutas cruzadas (GAP 22 %).

Iteración 25: lo social ordena sectores (GAP 16 %).

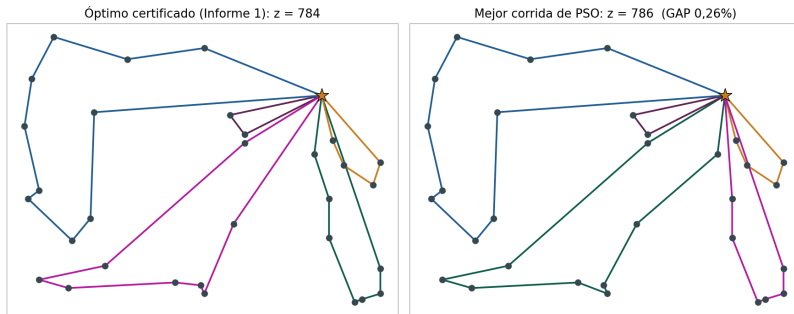
Iteración 500: rutas limpias (GAP 0,26 %).

Idea clave

El mejor global, **decodificado a rutas**, pasa de un ovillo aleatorio a 0,26 % del óptimo en $\approx 0,1$ segundos.

Rutas finales: óptimo certificado vs. mejor PSO

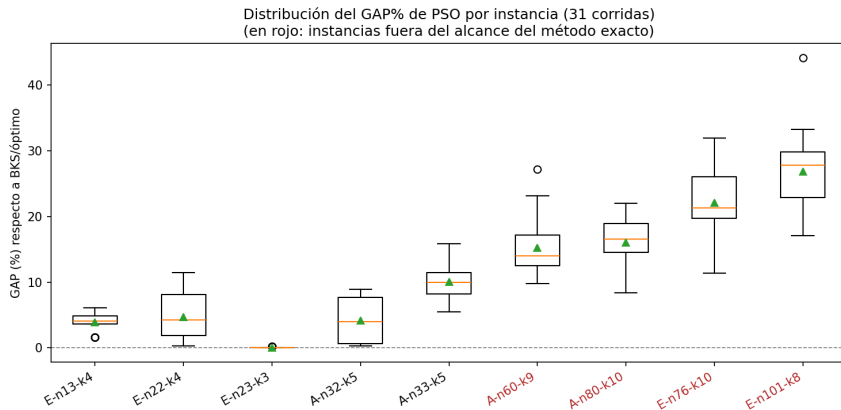
A-n32-k5: rutas del óptimo vs. la mejor solución de PSO



Idea clave

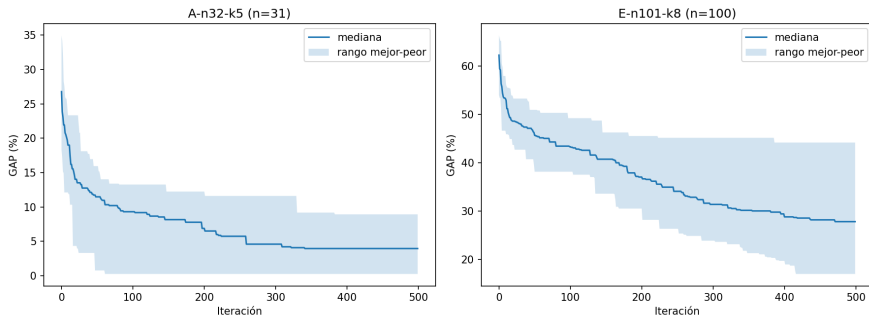
Rutas **casi indistinguibles**: misma sectorización, pequeñas diferencias de orden interno (784 vs. 786, GAP 0,26 %).

Calidad de solución: GAP % por instancia (31 corridas)



Compartidas ($n \leq 33$): GAP promedio 0,03 %–10,07 %, toca el óptimo en E-n23-k3. **Escalabilidad** ($n \geq 60$): 15,25 %–26,84 %, crece al salir del rango de tuning.

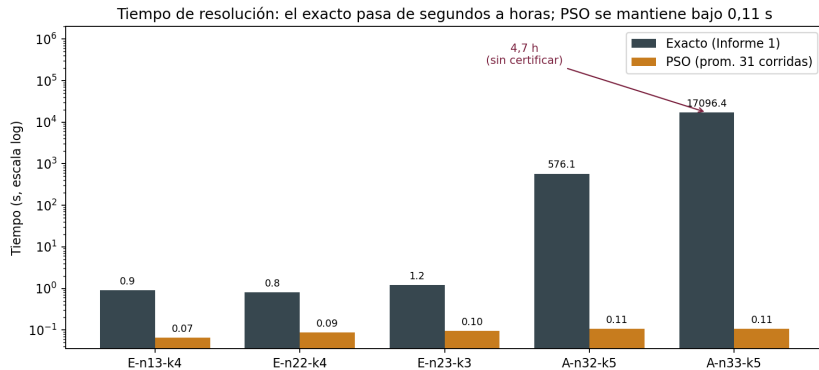
Convergencia: el presupuesto sobra en las chicas, apenas alcanza en las grandes



Idea clave

En A-n32-k5 la mediana deja de mejorar en la iteración **341** de 500; en E-n101-k8 sigue mejorando hasta la **471**.

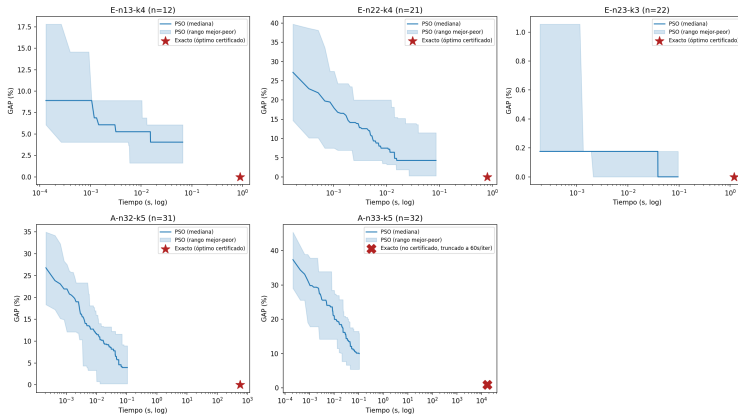
Tiempo de resolución: el acantilado del método exacto



Idea clave

Mismo equipo para ambos métodos. En A-n32-k5 PSO es $\approx 5\,460\times$ más rápido; en A-n33-k5, $\approx 161\,000\times$.

La comparación completa: PSO converge antes de que el exacto despegue



El exacto se reporta como un **punto** (★ óptimo certificado, × truncado sin certificar). Eje de tiempo en **escala log**.

Conclusiones

Conclusiones: los dos métodos, cara a cara

Método exacto

- **Certifica optimalidad:** preferible sin ambigüedad mientras sea tratable.
- Pero la tratabilidad se pierde **de golpe:** $n=32 \rightarrow 33$ pasó de minutos a 4,7 h sin certificado.

PSO (SR-1)

- Sin certificado, pero **tiempo acotado y predecible** ($\leq 0,31$ s hasta $n=101$).
- $GAP \leq 5\%$ cerca del rango de tuning; hasta 27 % al alejarse de él.

Idea clave

La conclusión central es la **complementariedad**, no la sustitución: **el exacto certifica pero no escala; PSO escala pero no certifica.**

Conclusiones: hallazgos específicos de este trabajo

Más allá de la dicotomía general, los datos revelan cuatro observaciones concretas:

- 1 **El acantilado es abrupto, no una pendiente.** Un solo cliente más ($n=32 \rightarrow 33$) multiplicó el tiempo del exacto por ≈ 30 ($576\text{ s} \rightarrow 4,7\text{ h}$) y aun así **no certificó**: la intratabilidad aparece de golpe.
- 2 **Donde ambos aplican, certificar aporta poco a la calidad.** En 4 de las 5 instancias compartidas PSO quedó a $< 5\%$ del óptimo real, en *miles* de veces menos tiempo: el certificado del exacto vale por la garantía, no por una mejora tangible de calidad ahí.
- 3 **La dificultad de PSO no depende solo de n .** E-n76-k10 (GAP 22%) resultó más difícil que A-n80-k10 (GAP 16%) pese a ser más chica: pesa la **estructura** de la instancia (razón demanda/capacidad, familia), no solo su tamaño.
- 4 **El presupuesto fijo de evaluaciones es un arma de doble filo.** Sobra en instancias chicas (A-n32-k5 converge en la iteración 341/500) y falta en las grandes (E-n101-k8 aún mejora en la 471/500).

Conclusiones: recomendación y trabajo futuro

Idea clave

Recomendación práctica: usar el **método exacto** mientras la instancia caiga antes del acantilado ($n \lesssim 32$ en este banco), donde es rápido y certifica; a partir de ahí, **PSO** es la única de las dos opciones que devuelve una solución usable en tiempo acotado.

En el informe

Trabajo futuro (derivado directamente de los hallazgos anteriores):

- Variante **GLNPSO** con vecindario social ($lbest/nbest$) para cerrar la brecha de GAP frente a la literatura (hallazgo 2).
- **Escalar** el número de iteraciones con la dimensión $(n-1)+2m$, en vez de fijarlo por presupuesto (hallazgo 4).
- Ampliar el **tuning** a todo el rango de tamaños y estructuras evaluadas (hallazgos 3 y 4).

Referencias

Problema y método exacto (Informe 1)

Laporte, G., & Nobert, Y. (1983). A branch and bound algorithm for the capacitated vehicle routing problem.

Operations Research Spektrum, 5(2), 77–85.

Lysgaard, J., Letchford, A. N., & Eglese, R. W. (2004). A new branch-and-cut algorithm for the capacitated vehicle routing problem. *Mathematical Programming*, 100, 423–445.

Toth, P., & Vigo, D. (Eds.). (2002). *The Vehicle Routing Problem*. SIAM.

CVRPLIB (2024). *Capacitated Vehicle Routing Problem Library*. vrp.atd-lab.inf.puc-rio.br.

Contextos de aplicación

Golden, B. L., Assad, A. A., & Wasil, E. A. (2002). Routing vehicles in the real world: applications in the solid waste, beverage, food, dairy, and newspaper industries. En *The Vehicle Routing Problem* (pp. 245–286). SIAM.

Beliën, J., De Boeck, L., & Van Ackere, J. (2014). Municipal solid waste collection and management problems: a literature review. *Transportation Science*, 48(1), 78–102.

Metaheurística: PSO (Informe 2)

Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. *Proc. IEEE ICNN*, 1942–1948.

Shi, Y., & Eberhart, R. (1998). A modified particle swarm optimizer. *Proc. IEEE CEC*, 69–73.

Clerc, M., & Kennedy, J. (2002). The particle swarm: explosion, stability, and convergence. *IEEE Trans. Evol. Comput.*, 6(1), 58–73.

Poli, R., Kennedy, J., & Blackwell, T. (2007). Particle swarm optimization: An overview. *Swarm Intelligence*, 1(1), 33–57.

Ai, T. J., & Kachitvichyanukul, V. (2009). Particle swarm optimization and two solution representations for solving the CVRP. *Computers & Industrial Engineering*, 56(1), 380–387.

Herramientas

NumPy (Harris et al., 2020), *Numba* (Lam et al., 2015), *Optuna* (Akiba et al., 2019).

Iconos: Pidgey © Nintendo / Game Freak, uso académico ilustrativo; sobre, camión y cliente: bancos de imágenes de uso libre.

Gracias

¿Preguntas?

Cristopher Angulo Ahumada

Código y datos: github.com/OrejiNet/CVRP-Exact-Method-vs-Particle-Swarm-Optimization

Anexo: glosario de símbolos

Instancia y solución

$G = (V, E)$	grafo completo no dirigido
$V = \{1, \dots, n\}$	nodos (1 = depósito, resto clientes)
E	aristas (todos los pares de nodos)
n	cantidad de nodos, depósito incluido
m	cantidad de vehículos de la flota
D	capacidad de cada vehículo
d_i	demanda del cliente i
c_{ij}	costo (distancia) de la arista $\{i, j\}$
$\mathcal{R}$	conjunto de rutas de una solución
r	ruta: ciclo cerrado desde el depósito
z	objetivo: distancia total de la flota

Modelo entero (método exacto)

x_{ij}	veces que se usa la arista $\{i, j\}$
S	subconjunto de clientes (cortes RCI)
$ S $	cantidad de clientes en S
RCI	<i>rounded capacity inequality</i>

PSO

$\mathbf{x}_i$	posición de la partícula i
$\mathbf{v}_i$	velocidad de la partícula i
$\mathbf{p}_i$	mejor posición propia (<i>pbest</i>)
$\mathbf{p}_g$	mejor del enjambre (<i>gbest</i>)
N	tamaño del enjambre
T	cantidad de iteraciones
t	iteración actual, $t = 1, \dots, T$
w	inercia: decrece de w_0 a w_f
c_1, c_2	coeficientes cognitivo y social
$\mathbf{r}_1, \mathbf{r}_2$	vectores aleatorios en $[0, 1]$
$v_{\max}$	tope de velocidad (fracción del rango)
SR-1	representación de valor real de Ai & Kachitvichyanukul (2009): $(n-1) + 2m$ dimensiones

Métricas

GAP %	$100 (z - z^{\text{ref}}) / z^{\text{ref}}$
z^{ref}	óptimo certificado, o BKS si no hay
BKS	<i>best known solution</i>

Anexo (1/2): variables, objetivo y restricciones de grado

Variables de decisión

Formulación simétrica de dos índices de Laporte & Nobert (1983):

x_{ij} = veces que la solución usa $\{i, j\}$

No es una variable binaria de asignación cliente-vehículo: el modelo no distingue *qué* vehículo hace cada ruta, porque la flota es homogénea.

Función objetivo

Distancia total recorrida por la flota:

$$\text{mín } z = \sum_{\{i,j\} \in E} c_{ij} x_{ij}$$

Restricciones de grado

Depósito: salen y vuelven m vehículos, por lo que su grado es $2m$:

$$\sum_{j=2}^n x_{1j} = 2m$$

Cada cliente $k = 2, \dots, n$ se visita una sola vez, es decir, tiene grado 2:

$$\sum_{i < k} x_{ik} + \sum_{j > k} x_{kj} = 2$$

Dominio de las variables

$$x_{ij} \in \{0, 1\}, \quad x_{1j} \in \{0, 1, 2\}$$

para $i, j \in V \setminus \{1\}$, $i < j$. El valor 2 es la ruta que atiende a un solo cliente: se recorre la arista depósito-cliente de ida y de vuelta.

Anexo (2/2): capacidad y eliminación de subtours

Por qué no bastan los grados

Las restricciones de grado son **necesarias pero no suficientes**: admiten soluciones con ciclos desconectados del depósito. Para descartarlos e imponer a la vez la capacidad, se agrega, para todo $S \subseteq V \setminus \{1\}$ con $|S| \geq 2$:

$$\sum_{i \in S} \sum_{j \notin S} x_{ij} \geq 2 \left\lceil \frac{\sum_{i \in S} d_i}{D} \right\rceil$$

Si la demanda de S necesita $\lceil \cdot \rceil$ vehículos, deben cruzar su frontera al menos el doble de aristas.

Forma equivalente: RCI

En presencia de los grados, la desigualdad anterior equivale a la *rounded capacity inequality* (Lysgaard et al., 2004), que es la que se implementó:

$$\sum_{i,j \in S; i < j} x_{ij} \leq |S| - \left\lceil \frac{\sum_{i \in S} d_i}{D} \right\rceil$$

Familia de tamaño **exponencial** en n : por eso se genera **bajo demanda**, un corte por iteración, y no de antemano.

Idea clave

En PSO nada de esto se impone como restricción: el decodificador SR-1 **rechaza toda inserción que supere D** , así la **factibilidad sale por construcción**.

Apéndice: resultados completos de PSO (31 corridas por instancia)

Instancia	n	Ref.	Mejor	Prom.	Desv.	Gap prom. (%)	t (s)
E-n13-k4	13	247	251	256,6	3,0	3,89	0,066
E-n22-k4	22	375	376	392,5	12,3	4,66	0,086
E-n23-k3	23	569	569	569,2	0,4	0,03	0,096
A-n32-k5	32	784	786	816,7	25,4	4,17	0,106
A-n33-k5	33	661	697	727,6	16,2	10,07	0,106
A-n60-k9	60	1354	1486	1560,5	56,3	15,25	0,172
E-n76-k10	76	830	924	1013,2	40,3	22,07	0,212
A-n80-k10	80	1763	1911	2045,4	71,3	16,02	0,226
E-n101-k8	101	815	954	1033,7	44,2	26,84	0,306

Presupuesto: 50 000 evaluaciones por corrida; semillas 0, ..., 30. Núcleo de decodificación acelerado con Numba (~130 000–200 000 evaluaciones/s).

Apéndice: espacio de búsqueda de la parametrización

Parámetro	Rango explorado	Óptimo (Optuna)
N (partículas)	{20, 30, 50, 80, 100}	100
w_0	[0,5, 0,95]	0,9116
w_f	[0,1, 0,5]	0,2782
c_1	[0,5, 2,5]	2,4045
c_2	[0,5, 2,5]	1,1843
$v_{\text{máx}}$ (fracción)	[0,1, 1,0]	0,3762

Muestreador TPE; 60 *trials* ($\approx 10\times$ el número de hiperparámetros); $T = \lfloor 50\,000/N \rfloor$ para igualar esfuerzo entre configuraciones. Instancias de tuning: A-n34-k5 (GAP 3,34 %), A-n62-k8 (7,92 %), E-n76-k7 (14,37 %).